

实验：ReAct Agent 静态分析

1. 背景与目的

实现 **ReAct** 智能体：LLM 负责编排，**radare2** 与 **Ghidra** 作为可调用工具，对 **targets/challenge**（黑盒 ELF）做静态分析，由 Agent 给出漏洞结论。

不要求 exploit，不要求动态验证。

2. 目标程序

- 分析对象：**challenge**（Linux x86_64，已 strip，无源码）。
- 仅做静态分析；仅分析该文件。

3. 环境

- Ghidra、radare2
- LLM 须支持 Tool Calling（或等价可解析协议）；报告注明模型名称与日期。

4. 任务

1. 将 **r2** 与 **Ghidra** 封装为 Agent 只读工具（接口自定，须能支撑对 **challenge** 的静态分析）。
2. 实现 **Thought** → **Action** → **Observation** 闭环；Observation 仅来自工具返回。
3. 对 **targets/challenge** 运行 Agent，生成 **logs/run.txt**（完整交互日志）。

具体工具数量、轮次、prompt 由你设计；报告简要说明即可。

5. 输出（仅此三项）

(1) **vuln.json**

Agent 运行结束后写入项目根目录或 **output/vuln.json**，固定字段：

```
{  
  "vuln_type": "漏洞类型，如 stack_buffer_overflow",  
  "location": "Sink 所在函数或地址",  
  "cause": "一句话：不可信输入如何到达危险操作"  
}
```

须为 Agent **Final Answer** 的结构化结果；**不得**手工编造与日志无关的内容。

(2) logs/run.txt

完整 ReAct 日志；须能看出 **r2** 与 **Ghidra** 均被调用，且日志与 `vuln.json` **不矛盾**。

(3) Agent 源码

可一键运行；含 `requirements.txt` 与 Ghidra/r2 路径说明。

6. 提交

Git 单次 **commit** 推送；commit 说明含 **学号、姓名**。勿提交 API Key。并发邮件给我。